



Robotics & Computer Vision

Week 3 — Image Processing for Robotics Input

Robotics and Computer Vision · Hoa Sen University — This week we dive deep into how robots **see and understand the visual world**: from raw pixel arrays to geometric features that drive robot control commands.

Learning Objectives

By the end of this session, you will be able to build a complete image processing pipeline that extracts actionable pixel coordinates for a robot controller.

1

Image Fundamentals

Understand digital images, pixels, histograms, and image coordinate systems.

2

Filtering Techniques

Master point operators, linear filtering, and non-linear filtering methods.

3

Segmentation & Features

Apply segmentation, extract contours, compute centroid and area of objects.

4

Robot Coordinates

Extract target pixel coordinates (u, v) to feed into a robot control loop.

Week 3 in the Course Outline

Theory Topics

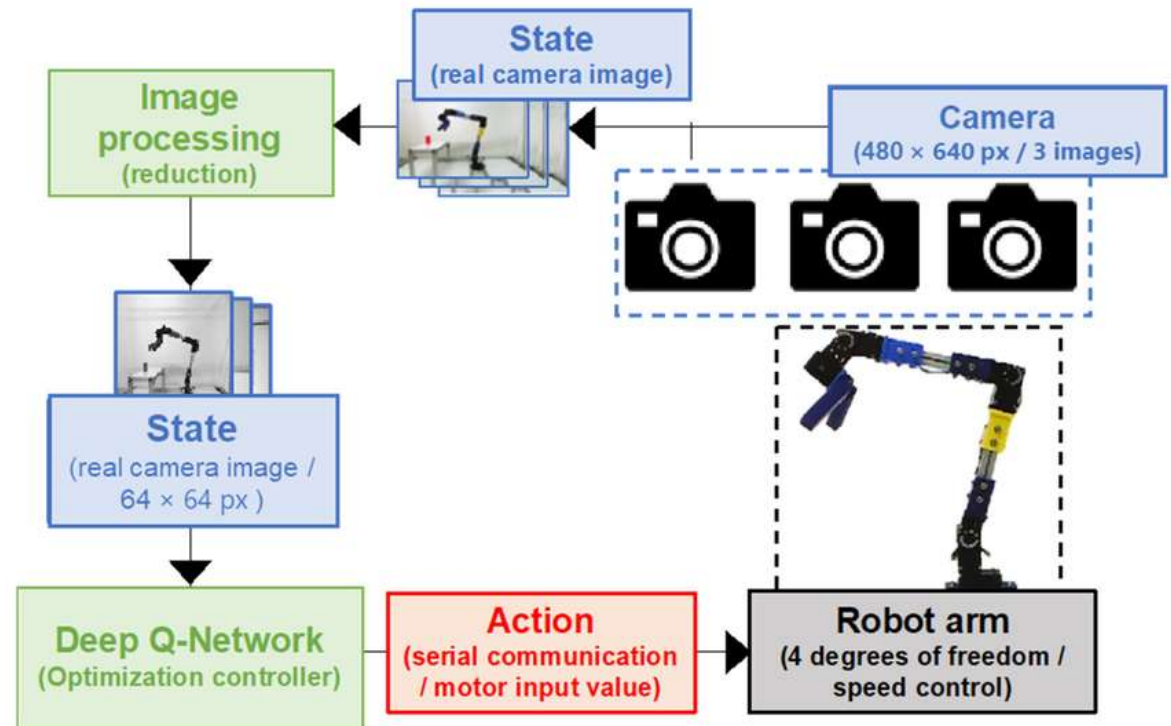
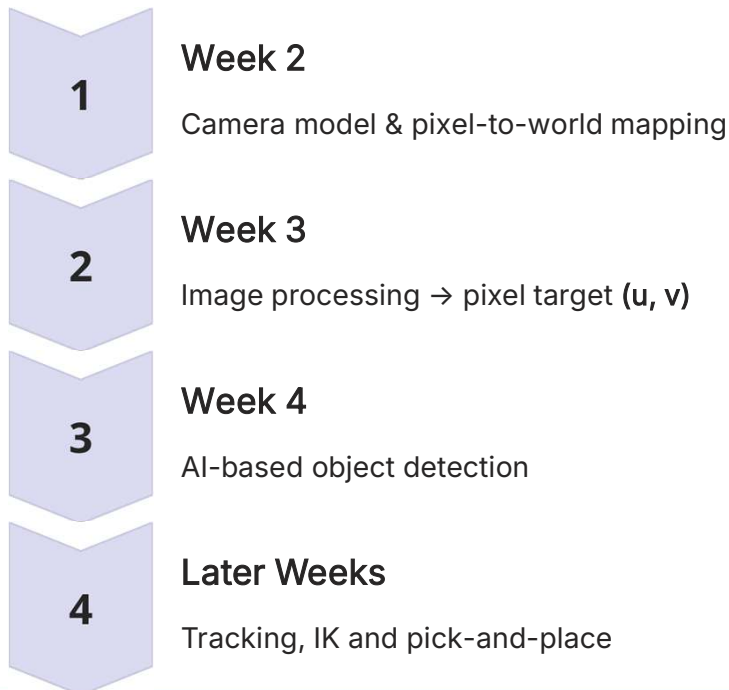
- Point operators — brightness, contrast, thresholding
- Linear filtering — convolution, mean, Gaussian, sharpening
- Non-linear filtering — median and morphological ops
- Fourier transforms for frequency-domain analysis
- Geometric transformations — rotation, scaling, warping

Practical Applications

- Image filtering and visual data cleaning
- Color-based segmentation in HSV space
- Finding contours, computing area, and locating object centroids
- Full OpenCV pipeline from webcam to (u, v) output

Connection to the Robotics Roadmap

Each week builds on the previous. Week 3 is the critical **vision-to-control bridge** — transforming raw pixels into actionable robot targets.



PART 1

Digital Images & Representation

The foundation you need before any processing or control can happen. Every robot vision system starts here.

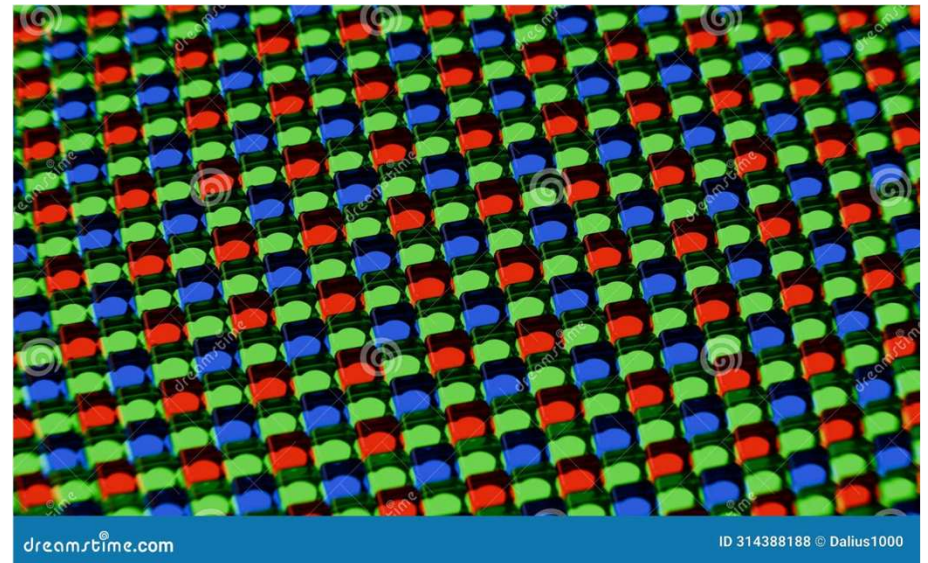


What Is a Digital Image?

A digital image is simply a **matrix of pixels**. Each pixel stores a light intensity value or color information. The resolution of an image determines how many pixels it contains — a 640×480 image holds 307,200 pixels.

Higher resolution means more detail, but also significantly higher computation cost. For real-time robotics, balancing resolution against processing speed is a key engineering decision.

📄 💡 In OpenCV, images are stored as **NumPy arrays** of shape (height, width, channels).



Grayscale vs. Color Images

Grayscale

Single channel with intensity values from **0 (black)** to **255 (white)**. Faster to process, ideal for thresholding and edge detection. Shape: (H, W).

Color (BGR)

Three channels: **Blue, Green, Red**. OpenCV reads images in BGR order by default — not RGB like most other libraries. Shape: (H, W, 3).

HSV

Hue, Saturation, Value. Separates color from brightness — much easier for robust color segmentation under varying light conditions.

Always be aware of which color space you are working in. A common bug is forgetting to convert from BGR to HSV before masking.

Image Coordinate System (u, v)

The image coordinate system places the **origin at the top-left corner** of the image. This is different from standard math coordinates where the origin is at the bottom-left.

u-axis (horizontal)

Increases from left to right. Corresponds to **column index** in the array.

v-axis (vertical)

Increases from top to bottom. Corresponds to **row index** in the array.

📌 ▲ In NumPy arrays, indexing is `img[row, col] = img[v, u]` — note the reversed order!

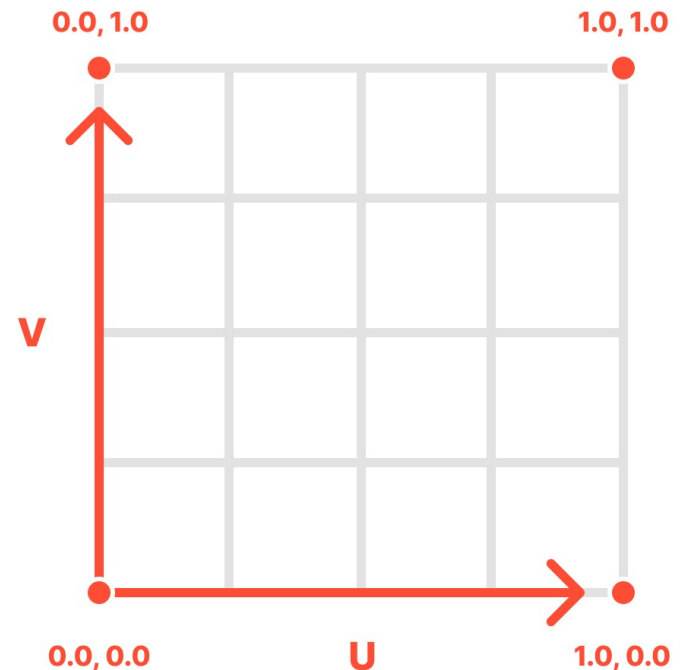
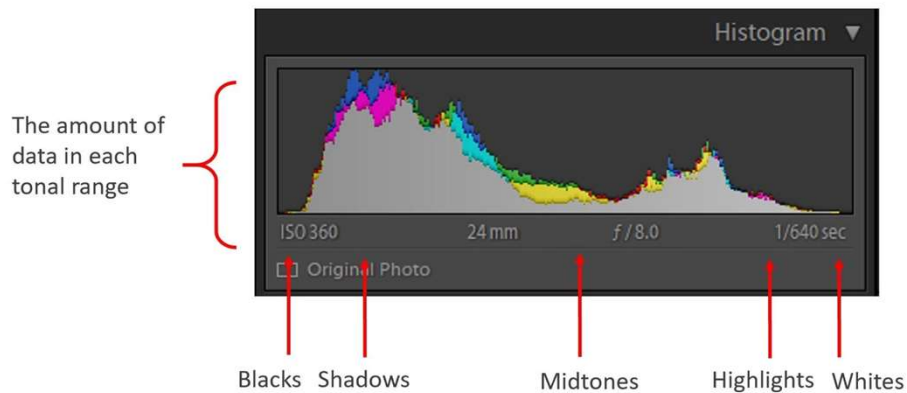


Image Histogram



A histogram plots the **frequency of each intensity value** (0–255) across all pixels. It is one of the most informative single-number summaries of an image's visual quality.

- **Dark image:** histogram peaks toward 0
- **Bright image:** histogram peaks toward 255
- **Low contrast:** histogram is narrow and clustered
- **High contrast:** histogram is wide and spread

Histograms are the foundation for thresholding, equalization, and evaluating whether your preprocessing pipeline is working correctly.

PART 2

Point Operators

Processing each pixel directly and independently — the simplest yet essential class of image operations.



What Is a Point Operator?

A point operator transforms each pixel value **independently**, without looking at neighboring pixels. The output depends solely on the input pixel's value and a fixed mapping function.



Fast & Easy to Implement

No neighborhood lookup required — just a direct value mapping, making these operations extremely efficient.



Common Examples

Brightness adjustment, contrast scaling, image inversion, gamma correction, and binary thresholding.



Limitation

Cannot handle spatially complex noise since each pixel is processed in isolation from its neighbors.

Brightness & Contrast Adjustment

Brightness

Add a constant β to every pixel: $\text{output} = \text{pixel} + \beta$. Positive β brightens, negative β darkens. Values are clipped to $[0, 255]$.

Contrast

Multiply by a factor α : $\text{output} = \alpha \times \text{pixel}$. $\alpha > 1$ increases contrast, $0 < \alpha < 1$ reduces it. Stretches the intensity range.

Combined

The general form is: $\text{output} = \alpha \times \text{pixel} + \beta$. This is OpenCV's `convertScaleAbs()` function — linear remapping in one step.

These are the simplest ways to normalize image quality before any further processing. Always check your histogram before and after applying these adjustments.

Thresholding

Thresholding converts a grayscale image into a **binary image** — pixels are either white (255) or black (0) based on whether they cross a threshold value.

→ Global Threshold

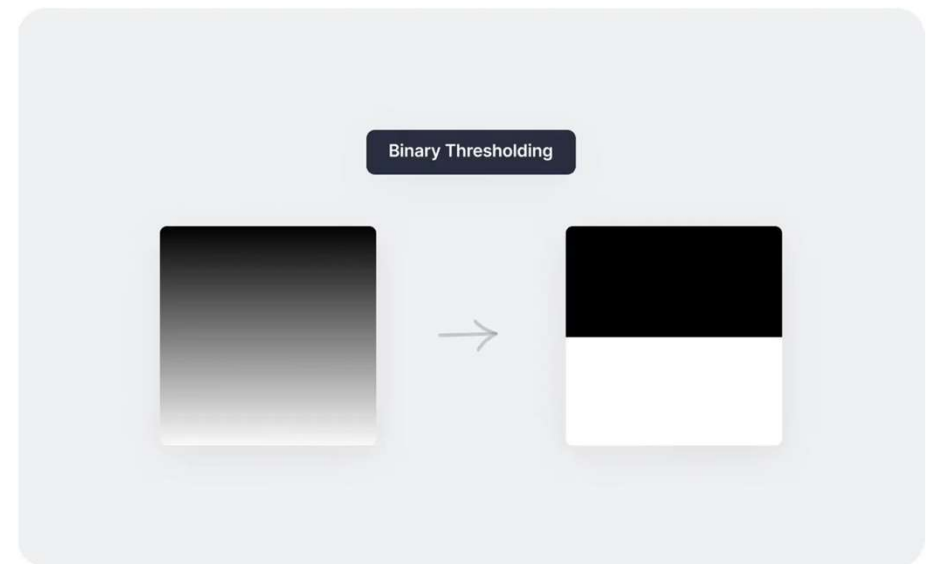
A single threshold T applied to the entire image: `if pixel > T → 255 else → 0`

→ Adaptive Threshold

Threshold varies across image regions, computed from local neighborhoods. Better for uneven lighting.

→ Otsu's Method

Automatically finds the optimal global threshold by minimizing intra-class variance of pixel intensities.



v7

Histogram Equalization

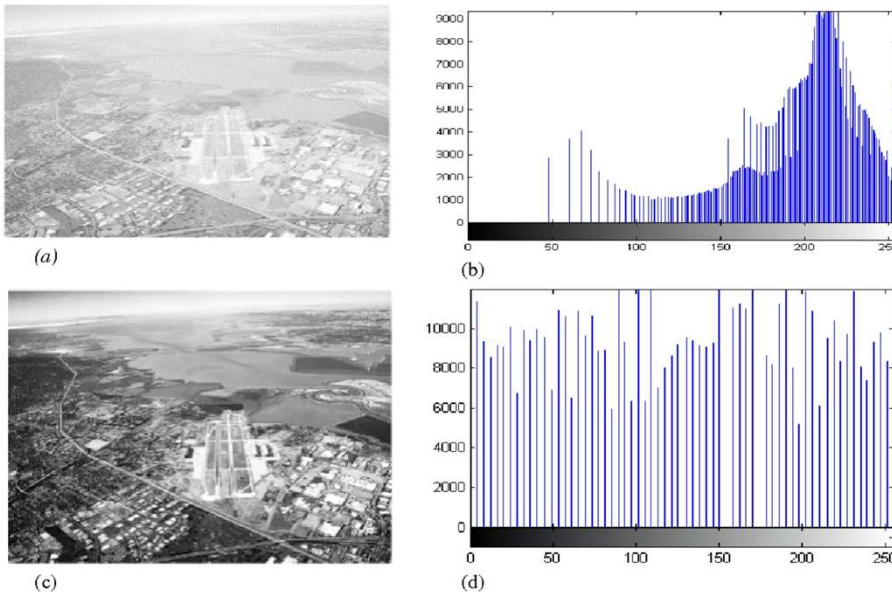


Fig. 1. (a) shows input image having low contrast. (b) shows histogram of input image. (c) shows

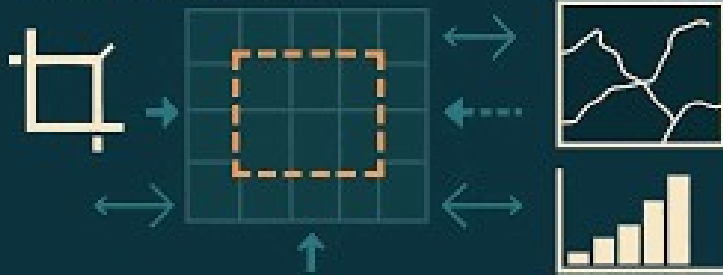
Histogram equalization **redistributes pixel intensities** to span the full range $[0, 255]$, dramatically improving contrast in dark or washed-out images.

- Useful for revealing hidden details in low-contrast images
- Can make fine textures more visible for analysis
- Risk: also amplifies noise — use with caution
- CLAHE (Contrast Limited AHE) is a gentler alternative

📄 ✅ Best used as a **controlled preprocessing step** — always validate visually before using in a robotics pipeline.

Image Processing

Resize, Crop, Filtering, Edge Detection
in Computer Vision



PART 3

Linear Filtering

Smoothing and transforming images through kernel-based convolution — the mathematical backbone of most image processing operations.

What Is Image Filtering?

Filtering creates a new pixel value based on the **neighborhood** of each pixel — not just the pixel itself. This enables far more powerful operations than point operators alone.



Noise Removal

Suppress random pixel variation from sensors.



Blurring

Reduce detail to simplify the image structure.



Sharpening

Enhance edges and fine details in the image.



Edge Finding

Detect boundaries between regions of different intensity.

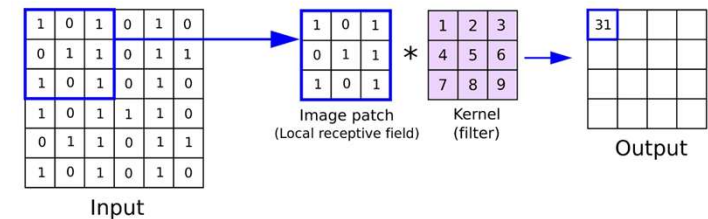
In **linear filtering**, the new pixel value is a weighted sum of the neighborhood — computed via convolution with a kernel matrix.

Convolution & Kernels

A **kernel** (or filter) is a small matrix — typically 3×3, 5×5, or 7×7 — that slides across the entire image. At each position, the kernel values are multiplied element-wise with the overlapping image patch, and the results are summed to produce a single output pixel.

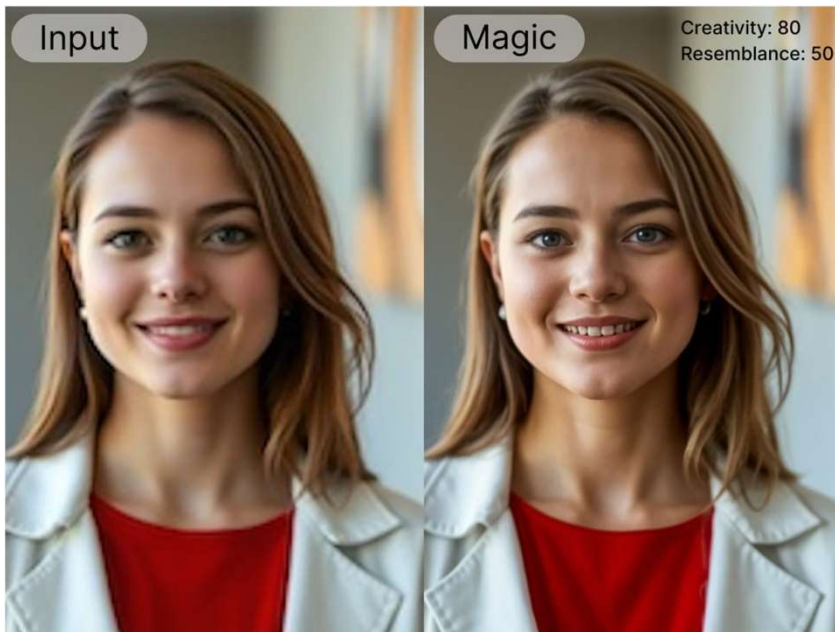
This operation is called **convolution** (or technically cross-correlation in most CV libraries). The kernel design entirely determines the effect:

- All-ones kernel → averaging / blur
- Gaussian-weighted kernel → smooth blur
- Derivative kernel → edge detection



Convolution kernels are also the core building block of **Convolutional Neural Networks (CNNs)** — the kernels in CNNs are learned from data.

Mean Filter



The mean (box) filter replaces each pixel with the **average value of its neighbors**. The kernel is a matrix filled with equal weights that sum to 1.

- Simple and fast to compute
- Reduces mild random noise effectively
- Significant drawback: **blurs edges strongly**, causing loss of important structural detail
- Useful mainly for teaching and conceptual illustration

In practice, Gaussian filter almost always outperforms mean filter for real robotics applications.

Gaussian Filter

The Gaussian filter applies a **weighted average** based on the Gaussian (bell curve) distribution — pixels closer to the center contribute more than distant ones.

Better Edge Preservation

The smooth weighting reduces over-blurring compared to the mean filter, preserving structural detail more faithfully.

Pre-processing for Canny

Gaussian blur is the mandatory first step before Canny edge detection — it suppresses high-frequency noise that would otherwise create false edges.

Most Practical Choice

This is the go-to smoothing filter for Week 3 lab work. Use
`cv2.GaussianBlur(img, (5,5), 0).`

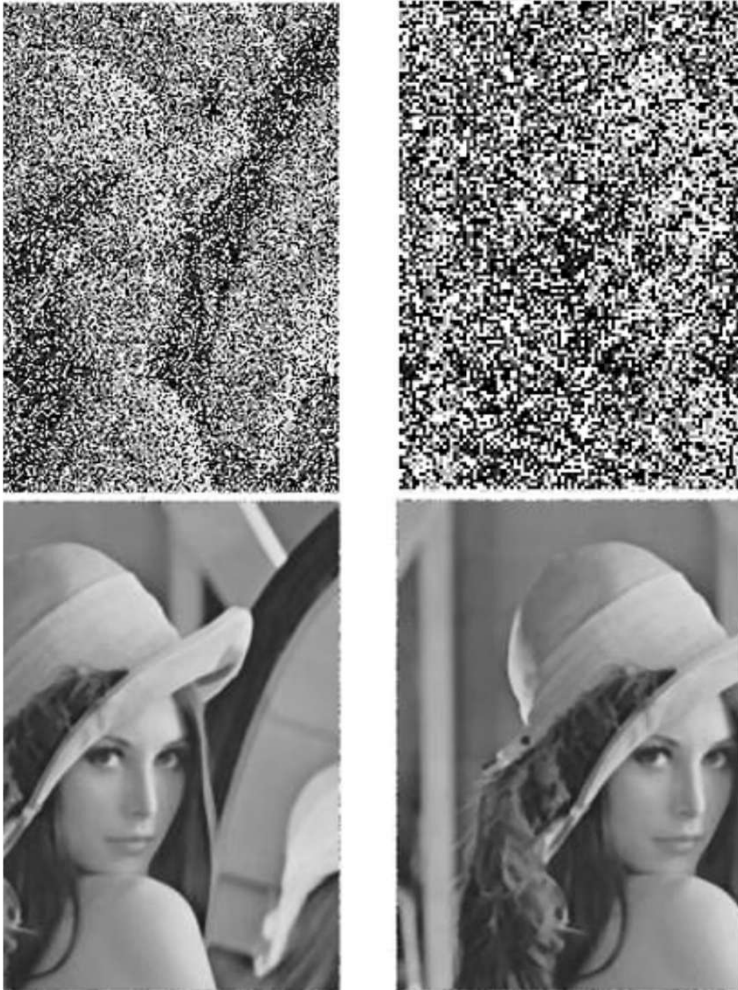
Sharpening & High-Pass Filtering

Sharpening filters **enhance edges and fine details** by amplifying high-frequency components (rapid intensity changes). A common approach is to subtract a blurred version from the original, then add the result back.

- Makes edges and textures appear crisper visually
- Serious risk: amplifies noise if the input is not already clean
- Rarely the right first step in a robot vision pipeline
- Consider only after denoising is complete

⚠ In robotics, clarity for **the algorithm** matters more than clarity for human eyes. A sharp-looking but noisy image can break segmentation.



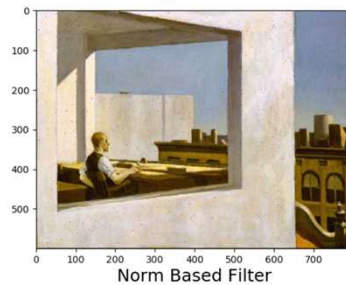
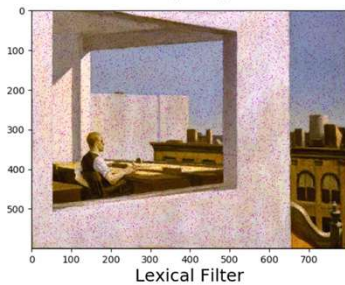
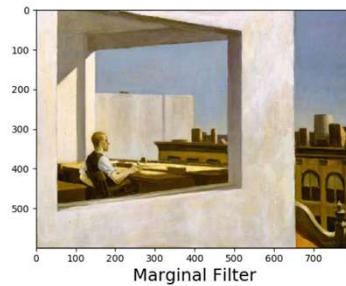
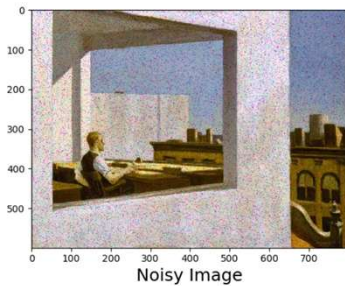


PART 4

Non-Linear Filtering

More powerful than linear filters for certain types of real-world noise — especially impulsive noise from sensors and cameras.

Median Filter



The median filter replaces each pixel with the **median value** of its neighborhood (sorted list), rather than the mean. This makes it highly resistant to extreme outlier values.

→ Eliminates Salt-and-Pepper Noise

Extreme black/white pixel spikes are replaced by the surrounding median — highly effective where linear filters fail.

→ Preserves Edges Better

Unlike mean filter, the median doesn't average across boundaries — edges remain sharper after filtering.

→ Ideal Before Segmentation

Apply median filter before color masking or contour detection to prevent noise from creating false contours.

Gaussian vs. Median: Which Filter to Choose?

Gaussian Filter

- Best for: mild random (Gaussian) noise
- Use before: Canny edge detection
- Behavior: smooth, continuous blurring
- OpenCV: `cv2.GaussianBlur()`

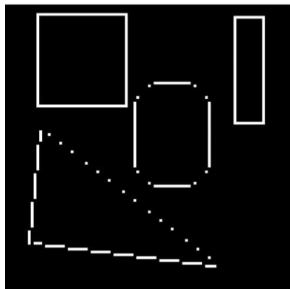
Median Filter

- Best for: impulsive / salt-and-pepper noise
- Use before: color segmentation & contours
- Behavior: non-linear, preserves edges
- OpenCV: `cv2.medianBlur()`

In a typical robot vision pipeline, you may use **both**: median filter first to remove impulsive noise, then Gaussian blur to smooth before edge detection. The right choice always depends on the type of noise present in your specific camera setup.



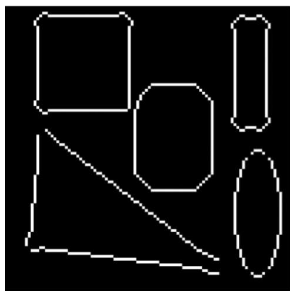
(a)



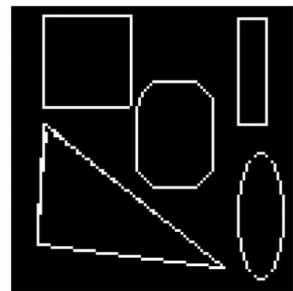
(b)



(c)



(d)



(e)

PART 5

Edge Detection

Finding the geometric boundaries of objects — essential for shape analysis, contour extraction, and structural understanding in robot vision.

What Is an Edge?

An edge is a location in an image where **pixel intensity changes rapidly**. Edges correspond to real-world boundaries between surfaces, objects, and lighting regions.



Encodes Shape Information

Edges carry the structural outline of objects — even without color or texture, edges alone can identify objects reliably.



Foundation for Contours

Contour detection (used later for centroid extraction) is built directly on edge maps from algorithms like Canny.



Helps Robots Understand Structure

Clear edges help robots recognize object orientation, size, and location — critical inputs for pick-and-place tasks.

Sobel & Canny Edge Detectors

Sobel Operator

Computes image gradient in the X and Y directions separately. Highlights horizontal or vertical edges depending on the kernel direction. Fast but sensitive to noise — best used after pre-smoothing.

```
cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

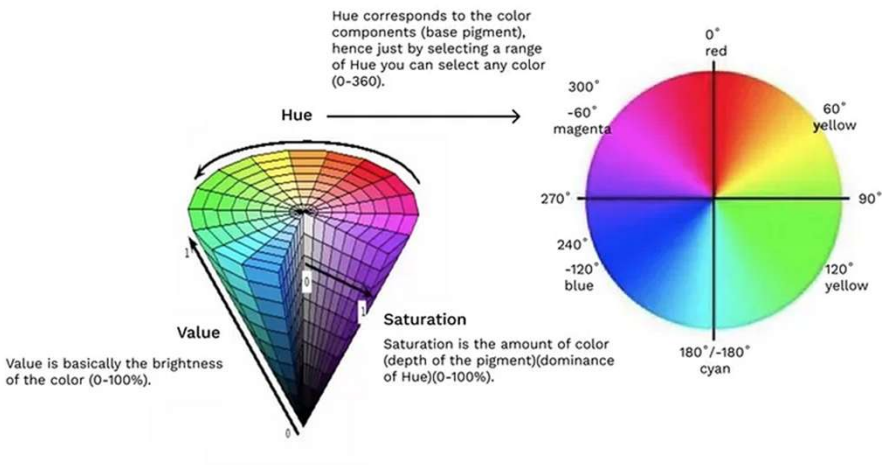
Canny Edge Detector

A multi-stage pipeline for robust edge detection:

1. Gaussian blur to reduce noise
2. Sobel gradient computation
3. Non-maximum suppression (thin edges)
4. Hysteresis double-thresholding

Canny produces **clean, single-pixel-wide edges** — the gold standard for classical computer vision.

HSV (Hue, Saturation and Value)



PART 6

Segmentation & Color Spaces

Separating the exact target object from the background — a critical step before feature extraction for robot control.

Why Use HSV Instead of BGR?

BGR (Blue-Green-Red) mixes color and brightness together, making it very difficult to define stable color ranges. The same red object can have wildly different BGR values under different lighting.

HSV separates these concerns:

H — Hue

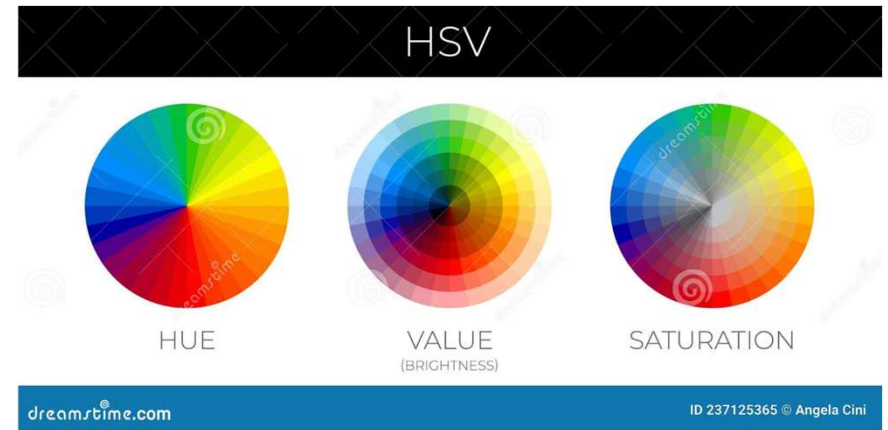
The actual color (0–180° in OpenCV). Red, green, blue, yellow each map to a specific range.

S — Saturation

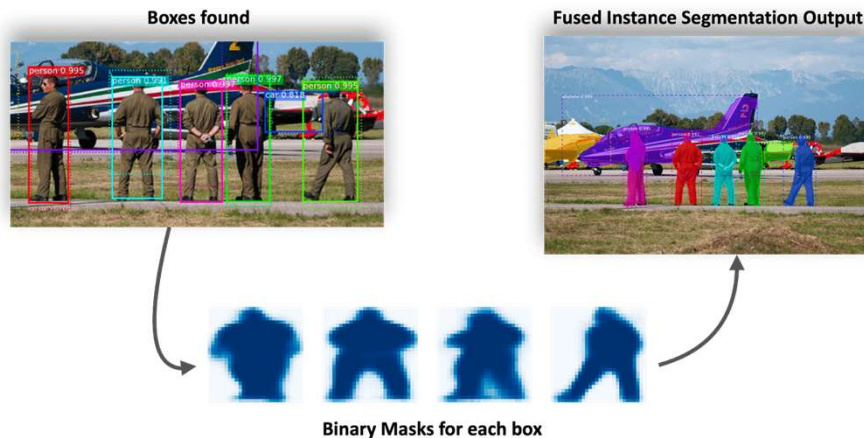
Color purity. Low S means washed out / gray. High S means vivid color.

V — Value

Brightness. Adjusting lighting changes V but leaves H and S relatively stable.



Binary Mask Creation



Once in HSV space, `cv2.inRange()` creates a **binary mask** by comparing each pixel against a defined lower and upper HSV bound.

- Pixels **within** the color range → white (255)
- Pixels **outside** the range → black (0)
- The mask is a grayscale image of the same size as the input
- This binary mask becomes the **direct input to contour extraction**

```
lower = np.array([20, 80, 80])
upper = np.array([40, 255, 255])
mask = cv2.inRange(hsv, lower, upper)
```

Morphological Operations

After creating the binary mask, small noise spots and gaps often remain. Morphological operations clean up the mask before contour detection.

Erosion

Shrinks white regions. Removes small isolated noise pixels and thin protrusions from the mask boundary. Use `cv2.erode()`.

Dilation

Expands white regions. Fills small gaps and connects nearby disconnected regions. Use `cv2.dilate()`.

Opening

Erosion followed by dilation. Removes small noise while preserving overall object shape. Best for cleaning external speckles.

Closing

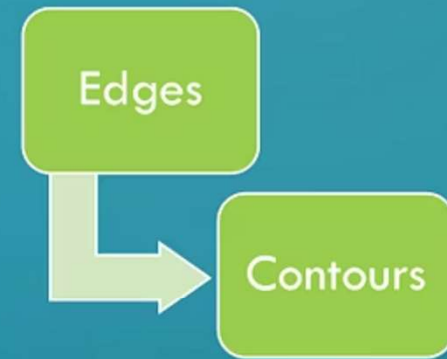
Dilation followed by erosion. Fills internal holes inside the object region. Best for filling gaps inside the mask.

PART 7

Contours & Geometric Features

Converting a binary mask into structured, quantitative features that a robot controller can actually use.

SECTION



/ silhouettes are basically edge features

edges – edges can also result from background

What Are Contours?

A contour is a **curve connecting all continuous boundary points** of an object region. In OpenCV, contours are extracted from binary masks or edge images as lists of (x, y) coordinate points.

- Each contour represents one connected object region
- Multiple contours may exist — pick the largest by area
- Foundation for computing area, perimeter, centroid, and bounding box
- One of the most practical tools in classical OpenCV pipelines

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```



Area & Bounding Box

Contour Area

Computed with `cv2.contourArea(cnt)`. Use area to **filter out small noise contours** — set a minimum area threshold and ignore contours below it. Prevents false detections from small color patches.

Bounding Box

`cv2.boundingRect(cnt)` returns (x, y, w, h). Draw with `cv2.rectangle()` for visualization. Also used as a **Region of Interest (ROI)** to speed up processing by limiting the search area.

Always filter contours by minimum area before computing centroids — even a 5-pixel noise patch can yield a contour and produce a misleading target coordinate.

Image Moments & Centroid

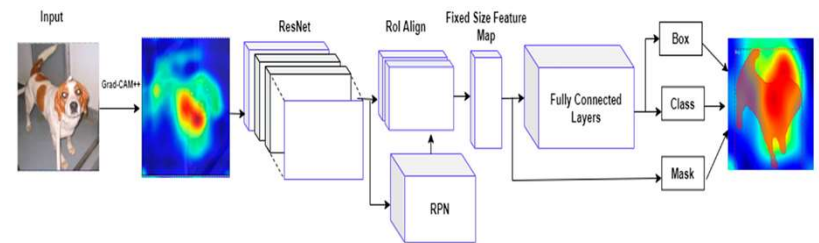
Image moments are weighted sums of pixel coordinates and intensities within a contour region. They encode the shape's geometric properties.

The **centroid** (geometric center of mass) is the most important output of Week 3. It tells the robot exactly where the object is located in the image frame:

Centroid Formula

```
M = cv2.moments(cnt)
u = M["m10"] / M["m00"]
v = M["m01"] / M["m00"]
```

Here **m00** is the total area, **m10** is the x-weighted sum, and **m01** is the y-weighted sum.



The Meaning of (u, v) for Robot Control

The centroid coordinates (u, v) are not just image data — they are the **control signal** that drives robot motion:

→ Object Shifted Left (u is small)

The robot or camera needs to move left to re-center the object in the frame. The error is $u_{\text{error}} = u_{\text{target}} - u_{\text{center}}$.

→ Pre-cursor to 3D Localization

(u, v) is the 2D target pixel. Combined with depth data or the camera model from Week 2, it maps to a full 3D world coordinate for grasping.

→ Object Too High or Low (v offset)

Vertical v deviation signals the robot to adjust its vertical position or arm angle accordingly.

→ The Vision-to-Control Bridge

This is the fundamental link between the vision system and the robot controller — the output of perception, the input of action.

PART 8

OpenCV Hands-On Practice

From code samples to a complete webcam-to-coordinates pipeline — putting everything together in the lab.

Lab 1: Reading Images & Color Space Conversion

The starting point for every OpenCV pipeline: load an image and convert to the color spaces needed for downstream processing.

```
import cv2

# Read image from disk (BGR format by default)
img = cv2.imread("object.jpg")

# Convert to Grayscale for thresholding / edge detection
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert to HSV for color-based segmentation
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Display result
cv2.imshow("Original", img)
cv2.imshow("Grayscale", gray)
cv2.imshow("HSV", hsv)
cv2.waitKey(0)
```

💡 Always verify your conversion with `cv2.imshow()` before piping into segmentation. A wrong color space will silently break the mask.

Lab 2: Blur & Color Segmentation

Apply Gaussian blur to reduce noise, then create a binary mask for the target color using HSV thresholding.

```
import numpy as np

# Step 1: Gaussian blur to suppress noise before segmentation
blur = cv2.GaussianBlur(img, (5, 5), 0)

# Step 2: Convert blurred image to HSV
hsv = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)

# Step 3: Define color range (yellow/orange object example)
lower = np.array([20, 80, 80])
upper = np.array([40, 255, 255])

# Step 4: Create binary mask
mask = cv2.inRange(hsv, lower, upper)

cv2.imshow("Mask", mask)
cv2.waitKey(0)
```

  Tune `lower` and `upper` HSV values using a trackbar UI to interactively find the best thresholds for your lighting conditions.

Lab 3: Contour Detection & Centroid Extraction

The complete pipeline — from cleaned mask to the final (u, v) target coordinate ready for robot control.

```
kernel = np.ones((5, 5), np.uint8)

# Clean mask with morphological ops
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

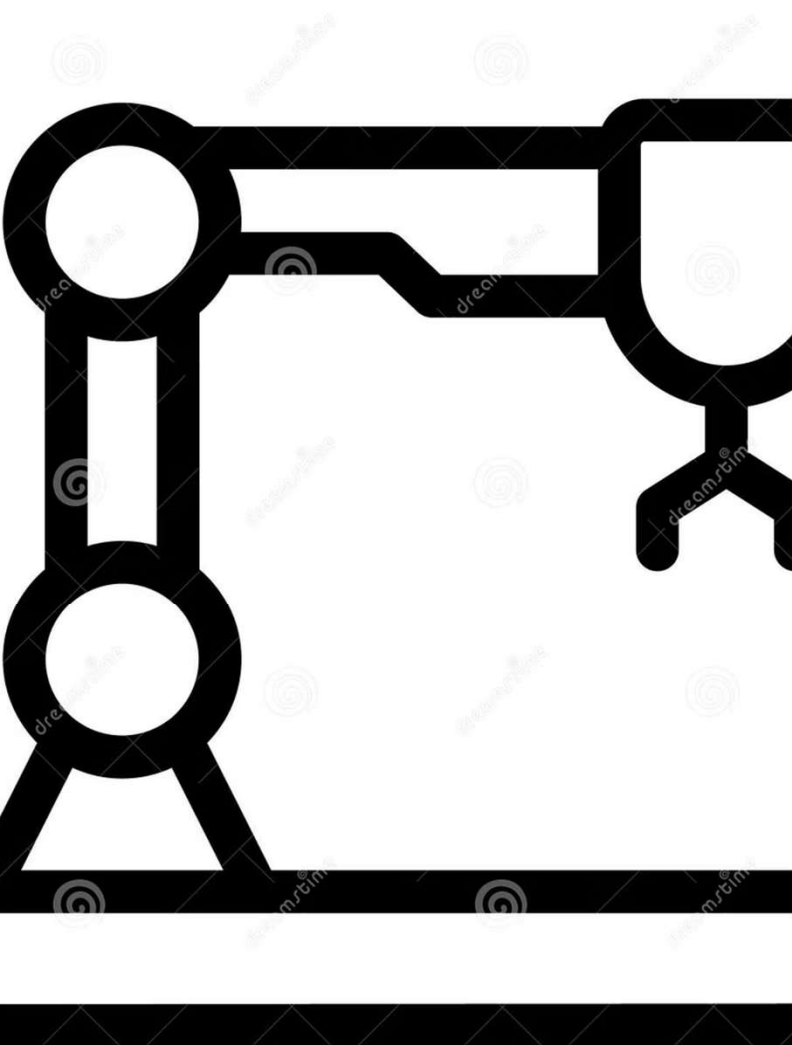
# Find contours in cleaned mask
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)

# Select largest contour (most likely the target object)
cnt = max(contours, key=cv2.contourArea)

# Compute image moments to get centroid
M = cv2.moments(cnt)
u = int(M["m10"] / M["m00"])
v = int(M["m01"] / M["m00"])

print(f"Target pixel: u={u}, v={v}")

# Draw centroid and bounding box
cv2.circle(img, (u, v), 5, (0, 255, 0), -1)
x, y, w, h = cv2.boundingRect(cnt)
cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
```



WEEK 3 COMPLETE

From Image to Robot Command

Week 3 gives you the complete toolset to extract a target pixel coordinate (u, v) from a live camera feed — the essential stepping stone toward AI vision and full robot control in the weeks ahead.



Assignment

Detect **2 different colors** simultaneously. For each: compute and display the contour area, centroid marker, and bounding box on the live webcam feed. Submit your annotated output image and code.



Next: Week 4

AI-powered object detection using deep learning models — moving from hand-crafted color rules to learned feature detection with YOLO and neural networks.